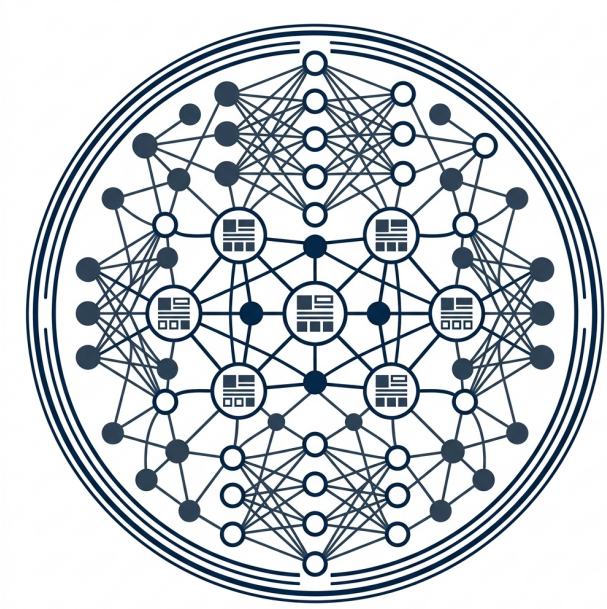
PROYECTO FINAL DE MINERÍA DE TEXTOS

WikiRAG-ES

Asistente RAG local sobre Wikipedia en español

Arquitectura agencial, recuperación híbrida y evaluación IR



AUTORES

Lucas González Fiz
Adan González Rodríguez
Santiago Palomino Vazquez
Javier Pérez Robles

Índice

1. Resumen del Proyecto **1**

2. Requisitos técnicos **1**
2.1. Módulo para cada temática 1
2.2. Archivos entregados 2

3. Entorno de Ejecución **2**
3.1. Tecnologías y librerías clave 2
3.2. Hardware utilizado en pruebas 2

4. Arquitectura del Sistema **3**
4.1. Diagrama de Flujo de Datos 3
4.2. Diseño Agencial del Router 3

5. Corpus de Conocimiento (MIRACL-es) **4**

6. Resultados de Evaluación IR **4**
6.1. Métricas obtenidas 4
6.2. Análisis: del baseline al sistema completo 4

7. Escalabilidad RAG según el Tamaño del Corpus **5**

8. Indexación Aproximada (HNSW) **6**

9. Benchmark de Cuantización (TurboVec) **7**

10. Comparativa de Técnicas Semánticas **7**

11. Ejemplos Reales del Asistente RAG **8**
11.1. Consulta Factual (QREL Experto) 8
11.2. Rechazo Informado (Prevención de Alucinaciones) 9
11.3. Llamada Agencial (Tool Routing) 9

12. Discusión y Limitaciones Conocidas **10**

13. Conclusiones **10**

A. Apéndice: Manual de Reproducción y Ejecución **11**

1. Resumen del Proyecto

Se ha diseñado e implementado un **asistente documental local** basado en la arquitectura de Recuperación Aumentada por Generación (*Retrieval-Augmented Generation*, RAG). El sistema descarga, normaliza y procesa un corpus masivo real (Wikipedia en español), fragmentándolo en *chunks* y generando representaciones vectoriales densas. A partir de estos datos, construye índices híbridos (FAISS y BM25) y ejecuta modelado de tópicos *offline*, permitiendo a un Agente LLM enrutar y responder consultas complejas en lenguaje natural.

- **Corpus:** 1 000 000 documentos de MIRACL-es (Wikipedia en español), con 200 consultas *qrels* de evaluación.
- **Escala de indexación:** 1 140 063 *chunks* procesados e indexados.
- **Motor LLM:** `gemma4:e4b` operando íntegramente en local a través de Ollama.
- **Recuperación:** Sistema híbrido que fusiona recuperación semántica (FAISS) y léxica (BM25) mediante *Reciprocal Rank Fusion* (RRF), con *reranking* opcional por *cross-encoder*.
- **Eficiencia:** Carga parcial de índices por modo de evaluación, ruta rápida para herramientas deterministas y cuantización vectorial TurboVec que reduce la memoria utilizada por los vectores en un 93,4 %.

2. Requisitos técnicos

2.1. Módulo para cada temática

Esta es la lista de los requisitos técnicos del proyecto.

Requisito	Módulo	Descripción
Embeddings	`indexing/embeddings.py`	Modelo Sentence Transformers multilingüe (384-dim).
Modelado de tópicos	`topics/modeling.py`	Agrupación semántica *offline* con BERTopic/NMF.
Resumen extractivo	`summarization/extractive.py`	TextRank sobre grafos + MMR para penalizar redundancia.
LLM Local	`llm/ollama.py`	Conexión nativa con Ollama, hiperparámetros ajustados.
Pipeline RAG	`retrieval/hybrid.py`	Fusión híbrida FAISS + BM25 mediante RRF, independiente de escalas de *score*.
Router Agencial	`agents/router.py`	Enrutado híbrido: atajos deterministas por regex y desambiguación con el LLM (salida JSON).
Innovación	`indexing/turbovec_store.py`	Compresión extrema de vectores mediante cuantización por anchura de bits (TurboQuant).

Cuadro 1: Mapeo de requisitos del enunciado y módulos desarrollados.

2.2. Archivos entregados

El proyecto se acompaña de los siguientes archivos, listos para su ejecución y auditoría:

Ruta del archivo o directorio	Contenido y función
`src/wikirag_es/`	Código fuente completo de la librería Python del paquete `wikirag-es`.
`data/processed/miracl_es_corpus_1M.jsonl`	Corpus local de 1 000 000 documentos con 200 consultas *qrels*.
`data/indexes/default/`	Índices FAISS (Flat) y BM25 generados sobre 1 140 063 fragmentos.
`data/reports/doc_eval_*.json`	Reportes de evaluación IR reproducidos para 50 consultas etiquetadas.
`data/reports/*bench*.json`	Resultados y métricas de los benchmarks empíricos de escalabilidad, latencia y compresión de memoria (HNSW, TurboVec).
`data/reports/miracl_es_corpus_package.tar.gz`	Entregable comprimido del corpus local, consultas de evaluación y manifiesto. El informe de tópicos se entrega como `data/reports/topics.json`.

Cuadro 2: Estructura de entregables del sistema.

3. Entorno de Ejecución

3.1. Tecnologías y librerías clave

Capa de software	Tecnología
Entorno y CLI	Python 3.12.13, Click 8.4.1
Modelado Vectorial	Sentence Transformers 5.5.1, modelo `paraphrase-multilingual-MiniLM-L12-v2` y PyTorch 2.5.1+cu121
Bases de Datos Vectoriales	FAISS CPU 1.14.2 (IndexFlatIP / IndexHNSWFlat), TurboVec (cuantización a 4 bits)
Recuperación Léxica y *Reranking*	BM25Okapi (`rank-bm25`) y cross-encoder `mmarco-mMiniLMv2-L12`
Motor de Inferencia	Ollama (servidor local), modelo `gemma4:e4b`
Procesamiento Analítico	BERTopic (UMAP+HDBSCAN), NetworkX (PageRank/TextRank)

Cuadro 3: Stack tecnológico del proyecto.

3.2. Hardware utilizado en pruebas

Para acelerar la generación de *embeddings* de más de un millón de fragmentos se ha aprovechado aceleración por hardware (CUDA).

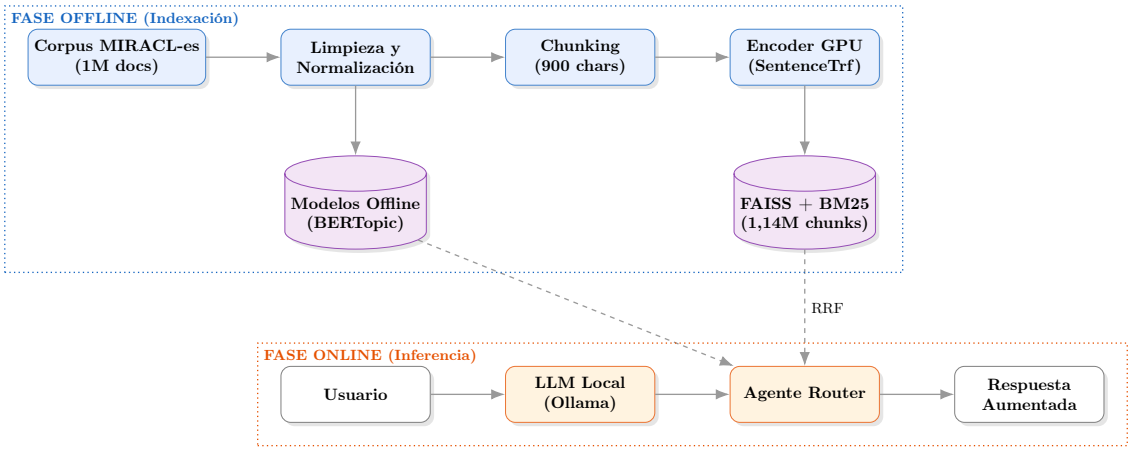
Componente	Especificación
Acelerador Gráfico (GPU)	NVIDIA GeForce RTX 5060 Ti (8 GB VRAM)
Motor Vectorial	PyTorch 2.12.0 con backend CUDA 13.0 habilitado y controlador NVIDIA 595.71.05 (CUDA 13.2)

Cuadro 4: Especificaciones del servidor local utilizado en la corrida documentada.

4. Arquitectura del Sistema

El flujo de trabajo se divide conceptualmente en dos grandes bloques: el procesamiento de datos por lotes (fase *offline*) y la interacción con el LLM en tiempo real (fase *online*).

4.1. Diagrama de Flujo de Datos



4.2. Diseño Agencial del Router

En lugar de construir un RAG tradicional donde toda pregunta activa incondicionalmente la base vectorial, se ha implementado un **Agente Router** de dos niveles. La clase `AssistantAgent` aplica primero atajos deterministas por expresiones regulares para disparadores explícitos (resumen, fecha, tópicos) y, ante consultas ambiguas, delega la decisión en el **LLM**, que selecciona la herramienta devolviendo un objeto JSON (`function-calling` ligero). Si el LLM no está disponible o su salida es inválida, el sistema recae con seguridad en la herramienta RAG. Esto ahorra tokens, reduce alucinaciones y mantiene un comportamiento robusto y trazable. El agente conserva además **memoria conversacional**: reescribe las preguntas de seguimiento como consultas autónomas antes de recuperar.

Herramienta	Patrón de activación	Ejecución algorítmica
`RagTool`	(Fallback por defecto)	Ejecuta FAISS y BM25, fusiona con RRF e inyecta fragmentos al LLM.
`SummaryTool`	*resume, ideas principales...*	Calcula TextRank sobre los documentos y devuelve un resumen exacto.
`TopicTool`	*tópico, tema, cluster...*	Extrae información del archivo JSON de clústeres de BERTopic.
`DateTool`	*qué día es hoy...*	Devuelve la fecha actual formateada en español (día de la semana incluido).
`WebSearchTool`	(Decisión del LLM)	Busca en la web (DuckDuckGo) información complementaria y la fusiona con el contexto local.

Cuadro 5: Inventario de herramientas del Agente Router.

5. Corpus de Conocimiento (MIRACL-es)

Se eligió MIRACL (*Multilingual Information Retrieval Across a Continuum of Languages*) como corpus principal por tres motivos fundamentales:

1. Contiene volcados directos de Wikipedia, asegurando alta densidad informativa.
2. Garantiza gramática y vocabulario nativo en español.
3. Proporciona conjuntos de evaluación manual (*qrels*) validados por expertos, indispensables para poder medir objetivamente el rendimiento del sistema RAG.

Métrica del Corpus	**Volumen final**
Documentos indexados	1 000 000 documentos MIRACL-es
Chunks resultantes indexados	1 140 063 fragmentos de texto
Consultas de evaluación (*qrels* expertos)	200
Tamaño persistido (FAISS + `embeddings.npy`)	3,50 GB de vectores
Tópicos offline entregados	BERTopic sobre 100 198 documentos, 1 413 filas de `topic_info`

Cuadro 6: Estadísticas del corpus de entrega tras la fase de ingestión.

6. Resultados de Evaluación IR

El rendimiento de recuperación se evaluó sobre una muestra de **50 consultas** de MIRACL-es (*dev*) con $k = 10$, comparando tres configuraciones del motor frente al mismo *ground-truth*.

6.1. Métricas obtenidas

Motor de Recuperación	**P@10**	**R@10**	**MRR**	**nDCG@10**
Motor Léxico (BM25)	0,072	0,341	0,264	0,248
Motor Semántico (FAISS)	0,112	0,451	0,451	0,379
Fusión Híbrida (FAISS+BM25, RRF)	**0,108**	**0,482**	**0,542**	**0,435**

Cuadro 7: Métricas sobre el corpus 1M de entrega (50 *queries*, $k = 10$).

6.2. Análisis: del baseline al sistema completo

La progresión por capas evidencia el aporte de cada componente sobre el *baseline* léxico:

- **Híbrido vs. baselines:** la *Reciprocal Rank Fusion* (RRF) mejora consistentemente el *Recall@10*, el MRR y el nDCG@10 respecto a BM25 y FAISS por separado. El léxico aporta coincidencias exactas y el semántico captura paráfrasis. Su fusión eleva el MRR a **0,542** y el nDCG@10 a **0,435**, demostrando que no solo se recuperan más documentos relevantes, sino que se posicionan de forma mucho más óptima en el ranking.
- **Coste de escala:** BM25 sobre 1 140 063 fragmentos es el cuello de botella de la evaluación. Por ello se implementó carga parcial de índices: el modo léxico no carga FAISS ni el modelo de embeddings, y los benchmarks no cargan textos de fragmentos cuando solo necesitan `doc_id`.
- **Lectura del MRR (0,542):** en la muestra documentada, el primer documento relevante aparece de media alrededor de la segunda posición. Es una métrica central para RAG porque el LLM atiende preferentemente a las primeras fuentes del *prompt*.
- **Precisión y *qrels*:** *Precision@10* se mantiene moderada porque MIRACL-es marca pocos documentos relevantes por consulta. Recuperar 10 implica un techo de precisión bajo por construcción, de modo que *Recall@10* y MRR son indicadores más informativos.

7. Escalabilidad RAG según el Tamaño del Corpus

Para justificar el uso de Big Data en arquitecturas RAG, se ha diseñado un benchmark de escalabilidad que evalúa cómo se comporta la calidad y el rendimiento del sistema híbrido al variar el volumen de documentos base. Se fragmentó el corpus local en colecciones de 100 000, 500 000 y la totalidad de 1 140 063 fragmentos.

Tamaño del Corpus (chunks)	P@10	R@10	MRR	Latencia Media
100 000	0,045	0,146	0,157	313 ms
500 000	0,085	0,370	0,373	1546 ms
1 140 063	0,109	0,482	0,542	3520 ms

Cuadro 8: Evolución del desempeño en función de la cantidad de documentos procesados. (Recuperador Híbrido, 50 queries, k=10)

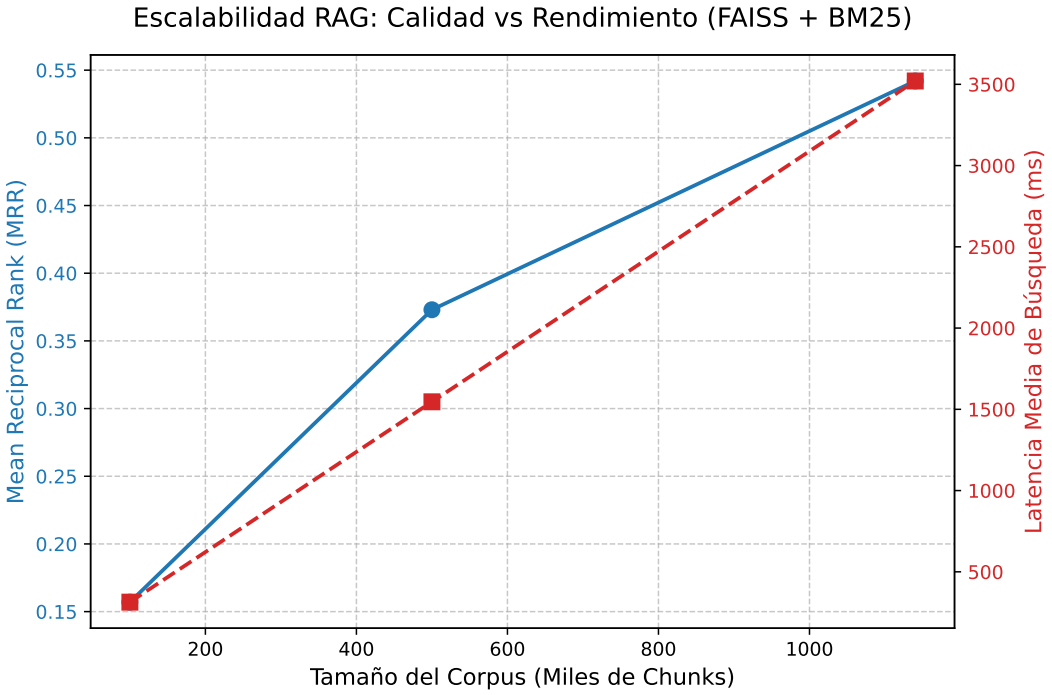


Figura 1: Relación entre calidad de recuperación (MRR) y degradación del rendimiento (latencia) al incrementar el tamaño del corpus.

Análisis de los resultados:

1. **Mejora de la calidad de recuperación:** Se observa empíricamente que, a medida que el modelo dispone de más texto fuente, la probabilidad de recuperar el fragmento correcto aumenta. Escalar de 100k a 1.1M *chunks* eleva el MRR de 0,157 a 0,542 y mejora significativamente el *Recall*.
2. **Impacto en el rendimiento:** Como era de esperar, la latencia total de la recuperación crece linealmente $O(N)$, alcanzando un promedio de más de 3.5 segundos por consulta al procesar el millón de documentos. Cabe destacar que el principal cuello de botella de esta métrica proviene del motor léxico BM25, ejecutado en Python puro, mientras que la parte semántica vectorial puede optimizarse fuertemente.
3. **Justificación arquitectónica:** Este aumento en la latencia y en el consumo de memoria al evaluar grandes volúmenes de documentos justifica la integración de las optimizaciones descritas en las siguientes secciones: la **Indexación HNSW** (para reducir el tiempo de búsqueda semántica en FAISS) y la cuantización **TurboVec** (para optimizar el uso de memoria RAM).

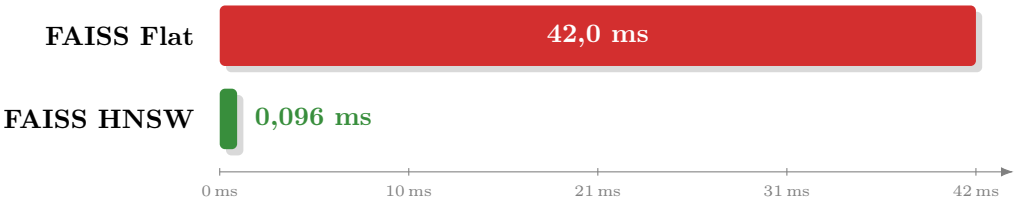
8. Indexación Aproximada (HNSW)

La búsqueda semántica exacta (`IndexFlatIP`) recorre 1 140 063 vectores en cada consulta, con coste lineal $O(N)$. Para reducir esa latencia se implementó una variante aproximada con **HNSW** (*Hierarchical Navigable Small World*), activable mediante `-index-type hnsw`.

HNSW representa los embeddings como un grafo navegable. En vez de comparar la consulta con todos los vectores, explora vecinos prometedores y converge rápidamente hacia zonas relevantes del espacio semántico. En FAISS se configuró con `IndexHNSWFlat`, `M=32`, `efConstruction=200` y `efSearch=64`, priorizando una búsqueda rápida sin modificar los embeddings originales.

Tamaño del índice	FAISS Flat	FAISS HNSW	Reducción
100 000 vectores	0,983 ms	0,100 ms	9,8x
500 000 vectores	16,178 ms	0,135 ms	119,8x
1 140 063 vectores	42,035 ms	0,096 ms	437,9x

Cuadro 9: Comparativa de latencia semántica exacta frente a HNSW. Resultados obtenidos con `scripts/bench_scaling_techniques.py`, 200 consultas y $k = 10$.



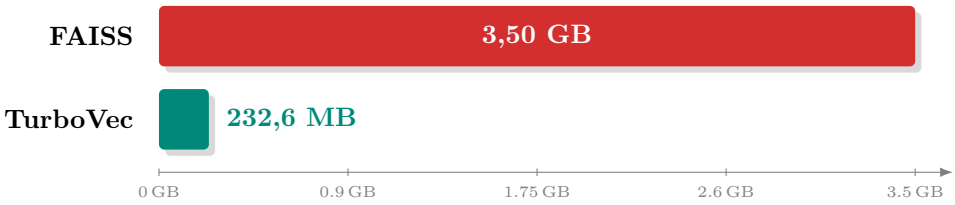
HNSW es útil cuando el cuello de botella principal es el tiempo de respuesta. En la medición aislada de búsqueda semántica sobre 1,14 millones de vectores, redujo la latencia de 42,0 ms a 0,096 ms. La contrapartida es que la búsqueda deja de ser exacta, por lo que `IndexFlatIP` se mantiene como referencia reproducible y HNSW como opción de escalado.

9. Benchmark de Cuantización (TurboVec)

Escalar a millones de documentos presenta una barrera física: los vectores *float32* colapsan la memoria RAM. Para solucionarlo, se implementó un motor secundario **TurboVec**, que cuantiza cada componente de los vectores a 4 bits (esquema TurboQuant de anchura de bits sobre un *IdMapIndex*).

Tipo de Índice vectorial	P@10	R@10	MRR	Memoria utilizada
Índice FAISS crudo (*float32*)	0,113	0,451	0,451	3,50 GB
Índice TurboVec Cuantizado (4-bit)	0,111	0,447	0,456	232,6 MB
Mejora de uso de memoria				**−93,4 %**

Cuadro 10: Comparativa de recursos semánticos (50 *queries*, $k = 10$). En la ejecución documentada, TurboVec redujo memoria de forma drástica. FAISS mantuvo menor latencia media en este benchmark de búsqueda persistida (54,0 vs. 77,3 ms/consulta).



TurboVec retiene el desempeño semántico casi a la perfección (caída de *Recall@10* inferior a medio punto) y reduce la memoria utilizada por los vectores de 3,50 GB a 232,6 MB (−93,4 %). Extrapolado a colecciones mayores, este factor de compresión es clave para desplegar el asistente en ordenadores convencionales.

10. Comparativa de Técnicas Semánticas

Una vez explicadas las dos optimizaciones principales, se comparó el tiempo de búsqueda puramente semántica (excluyendo BM25) para las tres arquitecturas indexadas implementadas en el proyecto: FAISS exacto, FAISS HNSW y TurboVec cuantizado.

La comparativa muestra dos estrategias complementarias:

- **FAISS HNSW:** reduce de forma extrema la latencia de búsqueda al sustituir el escaneo exhaustivo por navegación aproximada en un grafo. Es la opción más adecuada cuando la prioridad es responder rápido.
- **TurboVec Cuantizado (4 bits):** reduce de forma drástica la memoria persistida de los vectores y mantiene una calidad semántica muy parecida a FAISS exacto. Es la opción más adecuada cuando la limitación principal es almacenamiento o RAM.

FAISS exacto queda como referencia de calidad y reproducibilidad, HNSW como alternativa de baja latencia y TurboVec como alternativa de baja memoria. Según el escenario de despliegue, estas técnicas pueden elegirse de forma independiente o combinarse con la recuperación léxica BM25 y la fusión RRF del sistema híbrido.

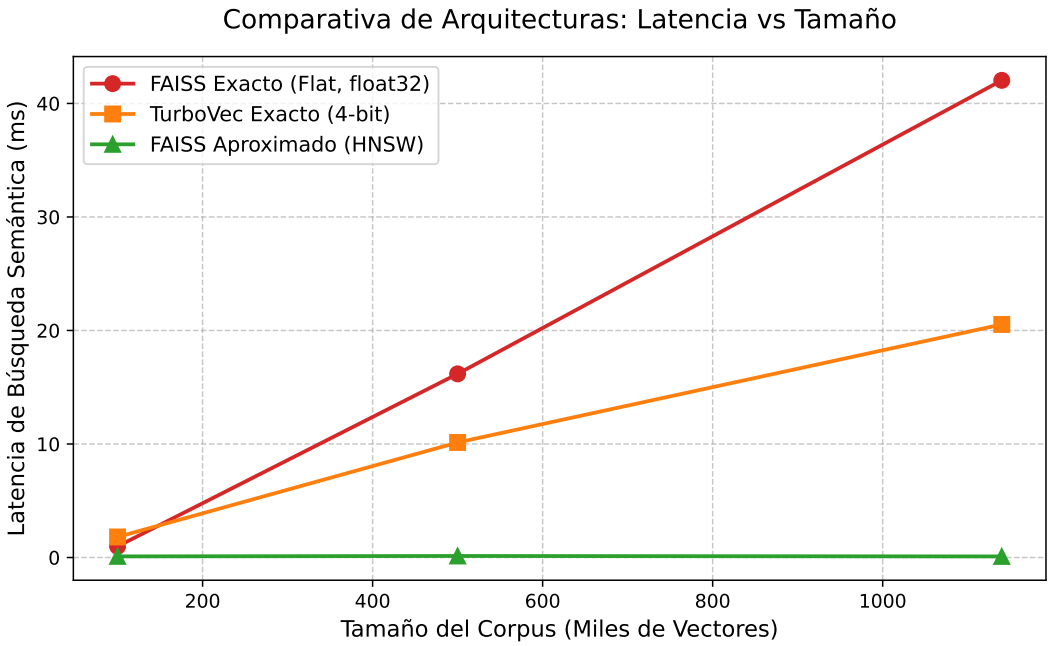


Figura 2: Escalabilidad de latencia por arquitectura semántica (FAISS Exacto vs FAISS HNSW vs TurboVec Cuantizado).

11. Ejemplos Reales del Asistente RAG

Las siguientes interacciones fueron generadas desde la interfaz CLI interactiva del proyecto (`wikirag chat`).

11.1. Consulta Factual (QREL Experto)

Usuario: *¿Cómo se convirtió Egipto en musulmán?* (consulta *QREL* oficial del *dev* de MIRACL-es).

> Tras la muerte de Mahoma en el año 632 comenzó la expansión musulmana y, en un lapso de cien años, los árabes ocuparon Egipto [Fuente 5]. Durante los primeros 300 años el islam fue ganando adeptos hasta convertirse en la religión predominante de la sociedad egipcia [Fuente 1]...

La respuesta es trazable y recupera fragmentos del artículo `es/33218`, marcado como relevante por los qrels expertos para esta consulta, confirmando la coherencia entre recuperación y ground-truth.
Usuario: *¿Qué país formó la primera Infantería de Marina?*

> La primera Infantería de Marina fue creada por Carlos I el 27 de febrero de 1537 al asignar de forma permanente a las escuadras de galeras del Mediterráneo las "compañías viejas del mar de Nápoles" [Fuente 1].

El modelo responde con precisión histórica basándose en `es/63108#3`. La evaluación completa queda en `data/reports/miracl_es_eval.jsonl`, con 200 preguntas validadas.

11.2. Rechazo Informado (Prevención de Alucinaciones)

Usuario: *¿Cuál es el horario de apertura del Museo del Prado?*

> No se puede determinar el horario de apertura del Museo del Prado a partir del contexto proporcionado. Las fuentes ofrecen información sobre la ubicación, historia y obras del museo, pero no incluyen datos sobre sus horarios [Fuente 1, 3, 7].

Se demuestra cómo el prompt agresivo ("No inventes datos") prohíbe exitosamente al LLM alucinar respuestas si los documentos recuperados de la Wikipedia no contienen horarios.

Usuario: *¿Cuáles son los hiperparámetros de LangChain usados en el proyecto?*

> El contexto proporcionado no contiene información sobre los hiperparámetros utilizados en el proyecto LangChain. Solo describe términos y conceptos relacionados con diversos campos como arquitectura, trazabilidad, tecnología y series de televisión [Fuente 1-8].

El sistema rechaza responder con firmeza. Al buscar sobre LangChain en Wikipedia recupera conceptos técnicos aleatorios, pero al no encontrar la respuesta exacta, el LLM declara explícitamente su ignorancia. Este comportamiento de ***cero-alucinación*** *es lo ideal en un RAG.*

11.3. Llamada Agencial (Tool Routing)

Usuario: *¿Qué día es hoy?*

> Hoy es sábado, 30 de mayo de 2026.

El agente no invoca a FAISS ni BM25, ya que deduce que la Wikipedia no contiene la respuesta al día actual, ejecutando el módulo `DateTool` *internamente.*

Usuario: *Resume el texto que tengas sobre la Revolución Industrial*

> *[Resumen generado por TextRank]*
> Revolución Industrial. La industria textil también fue la primera en utilizar métodos de producción modernos. (...) Se inició en la segunda mitad del siglo XVIII en el Reino Unido y se extendió unas décadas después a gran parte de Europa occidental y Norteamérica y concluyó entre 1820 y 1840.

Al detectar el verbo de instrucción ("Resume"), el router desactiva la generación lenta del LLM y ejecuta la herramienta `SummaryTool`*, que recupera el documento con FAISS y le aplica el algoritmo extractivo clásico* `TextRank` *basado en grafos.*

12. Discusión y Limitaciones Conocidas

1. **Cobertura parcial de Wikipedia:** El experimento documentado se ejecutó sobre 1 000 000 documentos MIRACL-es, un subconjunto amplio pero todavía inferior a los 10,4 millones disponibles en la colección completa. Si el usuario pregunta por un artículo que no está en el subconjunto, el motor recuperará el documento más similar posible y el LLM deberá declarar falta de información cuando el contexto no baste.
2. **Límites de memoria del Motor Léxico (OOM):** `rank-bm25` instancia las frecuencias léxicas directamente en RAM, por lo que la búsqueda léxica no escala de forma indefinida. A escala de decenas de millones de documentos se requeriría un motor invertido en disco (p. ej. Elasticsearch/Lucene).
3. **Cuello de botella en la latencia Híbrida (BM25):** A pesar de utilizar índices ultrarrápidos como HNSW (FAISS) o TurboVec para la búsqueda semántica, la latencia total por consulta del agente en modo híbrido ronda los 4 segundos. Esto se debe a que siempre se debe esperar a los resultados de la búsqueda léxica (BM25), la cual tarda casi 4 segundos en iterar sobre 1.14 millones de documentos en Python puro. La lentitud general observada no es atribuible a los modelos vectoriales aproximados o cuantizados (que resuelven en $\sim$1 ms), sino a la etapa léxica obligatoria para la fusión RRF.
4. **Latencia de Inferencia del LLM Local:** Independientemente de la velocidad de recuperación de documentos (que se resuelve en milisegundos usando FAISS o TurboVec), la interacción completa del chat añade una latencia significativa de entre 30 y 50 segundos. Este cuello de botella no radica en la recuperación de información, sino en el procesamiento del contexto inyectado (miles de tokens extraídos de los documentos de Wikipedia) y la subsiguiente generación de la respuesta elaborada palabra por palabra por parte del modelo `gemma4:e4b` ejecutado en un ordenador normal.

13. Conclusiones

El sistema desarrollado en un asistente documental funcional que integra embeddings, tópicos, resumen extractivo, LLM, RAG y agentes. A través de pruebas, se comprobó que la Recuperación Aumentada (RAG) depende críticamente de la calidad del recuperador y de la trazabilidad de las fuentes. Gracias a la compresión vectorial (TurboVec), la carga parcial de índices y el enrutamiento agencial nos permiten tener un sistema más eficiente sin sacrificar demasiada precisión.

A. Apéndice: Manual de Reproducción y Ejecución

Para facilitar la corrección y evaluación de esta práctica, se detalla a continuación el flujo completo de comandos necesarios para replicar los resultados desde la terminal. Toda la infraestructura se maneja mediante el CLI unificado `wikirag`.

A. Generación del Sistema desde Cero (1 Millón de Documentos)

```
# 1) Descargar corpus + qrels de validación
PYTHONPATH=src wikirag acquire-corpus \
  --corpus-output data/processed/miracl_es_corpus_1M.jsonl \
  --eval-output data/reports/miracl_es_eval.jsonl \
  --limit 1000000 \
  --eval-limit 200

# 2) Construir Índice Híbrido (FAISS Exacto + BM25)
PYTHONPATH=src wikirag build-index \
  --corpus data/processed/miracl_es_corpus_1M.jsonl \
  --index-dir data/indexes/default \
  --index-type flat

# 3) Generar Modelo de Tópicos Offline
PYTHONPATH=src wikirag topics \
  --corpus data/processed/miracl_es_corpus_1M.jsonl \
  --output data/reports/topics.json \
  --limit 100198
```

B. Arranque del Agente RAG Conversacional

Se requiere tener un servidor Ollama activo de fondo con el modelo `gemma4:e4b`.

```
PYTHONPATH=src wikirag chat \
  --index-dir data/indexes/default \
  --topics-report data/reports/topics.json
```

C. Reproducción de Benchmarks de Escalabilidad

Los scripts automatizados que generan la gráfica de degradación léxica y la gráfica comparativa de técnicas de vectorización (Secciones 7 y 7.1) pueden ser invocados directamente si el corpus base está construido:

```
PYTHONPATH=src python scripts/bench_scaling.py
PYTHONPATH=src python scripts/bench_scaling_techniques.py
```